Assignment

Information Retrieval / tester brief

You test a ranker against a simulated retrieval environment. A simulator in a Docker image serves a 100,000-item electronics catalogue. You write the ranker on the host and upload a ranking parquet file through a REST API. The simulator returns action logs and aggregate reports. After start, `/v1/spec` and `/docs` give the full HTTP schema.

What you receive

- Image tar `ire-a2-tester-api.tar` (about 1.3 GiB), tag `ire-a2-tester-api:latest`.
- The public data bundle, from `GET /v1/bundle` once the container runs.

Click to open OneDrive folder. Load the tar that matches the host (arm64 or amd64). World values from `/v1/spec`: `w908832361965e48e`, 100,000 items, 400 users, 144 queries, slate 20, rank 0 top, 3 graded rounds, about 14k `(query, user)` pairs per round. The simulator uses about 3.5-4 GiB RAM.

Load, start, and health

```
docker load -i ire-a2-tester-api.tar
mkdir -p work
docker run --rm -d --name ire-a2-tester \
  -p 127.0.0.1:8000:8000 \
  -v "$(pwd)/work:/work" \
  --memory=8g \
  ire-a2-tester-api:latest
```

The first start takes about 27-39 seconds while the world loads. Wait for health.

```
BASE=http://127.0.0.1:8000
curl -s $BASE/health
curl -s $BASE/v1/spec
```

`status` must be `ok`. `/v1/spec` gives `world_id`, slate size, rounds, and submission columns. `/docs` is the live endpoint schema.

Public bundle and schemas

Download the bundle with `curl -s $BASE/v1/bundle -o bundle_v1.tar.gz`, then unpack it with `tar -xzf bundle_v1.tar.gz`.

Read `bundle_v1/schemas.json` and `bundle_v1/student/README.md`, and check `world_id.txt` against `/v1/spec`. The bundle has catalogue text and embeddings, users and held-out users (segment plus eight numeric features), queries and query embeddings, a BM25 index, an embedding sidecar, an empty `bootstrap_log.parquet`, and `student/` helpers.

Ranker input and output

Traffic (`traffic_rN.parquet`): `world_id`, `round`, `impression_id`, `query_id`, `user_id`, `query_text`. **Submission** (`submission_rN.parquet`), exactly these columns: `world_id`, `query_id`, `user_id`, `rank`, `item_id`. Build one complete parquet for the published traffic of the round.

Validation rules

An invalid upload returns HTTP 422 with a capped list of errors. A valid file must:

- Use the `world_id` of the traffic file. The simulator refuses mixed worlds.
- Include every published `(query_id, user_id)` pair once, and no other pair.
- Give each pair 20 unique, valid catalogue `item_id` values.
- Use consecutive ranks 0-19, where rank 0 is the top of the slate.
- Have those five columns and no missing rows.

Three-round loop

A new session starts with **zero logs**. Run rounds 1, 2, 3 in order, at most 16 sessions per container. Below, `SID` is the returned `session_id` and `R` the round.

1. `POST $BASE/v1/sessions?seed=42` to create a session.
2. `GET $BASE/v1/sessions/$SID/rounds/$R/traffic` to download the traffic.
3. Produce a complete submission parquet with your ranker.
4. Upload it with `curl -X POST --max-time 1800` and `-F "file=@submission_r$R.parquet"` to `.../rounds/$R/submission`. A full-round submit for about 14k pairs holds high CPU for **more than 15 minutes**. Upload one file at a time.
5. `GET .../actions`, then `.../report`. Improve the ranker, then run the next round.
6. `GET $BASE/v1/sessions/$SID/report` for the session summary.
7. Optional: `DELETE $BASE/v1/sessions/$SID`. Session files stay in the mounted `work/` volume through container restarts.

`bootstrap_log.parquet` gives the action schema with zero rows, so the first behaviour data comes with the round 1 actions. Earlier action logs are extra training data for a later ranker. Each upload covers the traffic of its own round.

Action labels

Each served item gets one deepest label: `seen` (shown, no logged click), `click` (click, no later stage), `cart` (cart, no purchase), or `purchase` (purchase reached). A deeper label implies the earlier stages, and the log keeps the deepest one. Position and propensity bias the log, so actions are not ground-truth relevance.

What the report metrics mean

Round and session reports are **aggregates**. They give:

- **Expected value**: closed-form mean action value per impression, the stable score.
- **Sampled value**: value from the drawn actions in the log.
- **Baselines and deltas**: random, BM25, and semantic baselines, plus the round-to-round change.
- **Actions**: counts, rates, and exclusive reach per label.
- **Coverage**: how many catalogue items your slates used.
- **Head / middle / tail**: traffic-frequency bucket sizes.
- **Public user segments**: pair counts per segment. **Timing**: server simulate time.

A report has no per-item relevance and no oracle ordering, so learn from the action logs and read the aggregate score as a coarse check.

Feedback to send back

Write a short note, plus complete reproduction steps for each defect:

1. Setup problems: image load, startup, health wait.
2. Runtime and memory use.
3. Unclear schema or contract text.
4. Validation quality: a false accept, or 422 text that does not help.
5. Ranker development experience: bundle, traffic, wait time, score usefulness.
6. Bugs: commands, expected versus actual, evidence, severity (blocker / major / minor).

Stop and cleanup

Run `docker stop ire-a2-tester`. The image stays loaded. Send back the feedback note, the bug reproductions, your measured start time and memory use, one sample 422 if you hit validation, and the aggregate reports.